PhaseKit: Finite-Width Phase Diagrams for Neural Network Initialization with DAG-Based Variance Propagation and Formal Verification

halley-labs

Abstract

We present PhaseKit, a Python toolkit for neural network initialization diagnostics based on mean-field signal propagation. The tool computes phase labels, finite-width variance recursions, and graph-aware propagation summaries directly from PyTorch modules, and it exposes these analyses through a small developer-facing API. Beyond the core mean-field formulas, the current artifact includes DAG-based variance propagation, exploratory non-Gaussian corrections, theory-guided initialization heuristics, and SMT-backed algebraic sanity checks. Repository experiments support a positive but narrower claim than an accuracy leaderboard: the toolkit is useful for coarse initialization health checks, for tracing where phase predictions become fragile, and for surfacing architectures where mean-field assumptions break down. We therefore frame the experimental results as diagnostic evidence and case studies rather than as definitive head-to-head superiority claims over prior initialization methods.

1 Introduction

Deep neural networks are sensitive to initialization. Poole et al. [**?**] and Schoenholz et al. [**?**] established a mean-field theory showing that randomly initialized networks undergo phase transitions—*ordered*, *chaotic*, and *edge of chaos*—governed by the per-layer susceptibility $\chi_1 = \sigma_w^2 \mathbb{E}[\phi'(\sqrt{q^*}z)^2]$. When $\chi_1 < 1$, gradients vanish exponentially; when $\chi_1 > 1$, they explode; and when $\chi_1 = 1$, information propagates stably through depth. This theory underpins the Kaiming initialization [**?**] ($\sigma_w = \sqrt{2}$ for ReLU), but its predictions degrade for non-ReLU activations, deep networks, and finite widths.

PhaseKit operationalises and extends this theory. Given any PyTorch `nn.Module`, PhaseKit predicts the phase, recommends σ_w^* for criticality, and provides finite-width–corrected variance trajectories—all without training data or gradient computation. The key contributions are:

1. **Depth-calibrated phase classification** (§**??**): Per-layer χ_1 boundaries at $[10^{-4/L}, 10^{4/L}]$ that explicitly account for depth-dependent error accumulation.
2. $O(1/N^2)$ **finite-width corrections with empirical kurtosis** (§**??**): Second-order corrections incorporating sixth-moment terms, together with an explicit discussion of where Gaussian moment closure becomes biased.
3. **DAG-based variance propagation** (§**??**): A `torch.fx`-based engine with typed merge rules for residual, concatenation, and gating branches.
4. **Theory-guided initialization heuristics** (§**??**): practical initializers derived from mean-field criticality, orthogonal bases, and variance calibration, presented as exploratory utilities rather than settled winners in a baseline race.
5. **Python API** (§**??**): Three functions—`classify`, `initialize`, `diagnose`—cover common analysis workflows on arbitrary `nn.Module` instances.
6. **Z3 algebraic verification** (§**??**): SMT proofs of 20 algebraic properties (P1–P20) covering ReLU identities, convergence, and finite-width correction structure.

2 Mean-Field Theory Background

We review the mean-field theory of deep neural networks at initialization, following [**?**, **?**].

2.1 Variance Propagation

Consider a fully connected network with L layers, each of width N. At initialization, weights are drawn i.i.d. from $\mathcal{N}(0, \sigma_w^2/N)$ and biases from $\mathcal{N}(0, \sigma_b^2)$. The pre-activation at layer l for input $\mathbf{x}$ is:

$$h_i^{(l)} = \sum_{j=1}^{N} W_{ij}^{(l)} \phi(h_j^{(l-1)}) + b_i^{(l)} \tag{1}$$

where ϕ is the activation function. In the limit $N \to \infty$, the pre-activations become Gaussian with variance $q^{(l)}$ satisfying:

$$q^{(l)} = \sigma_w^2 \int \mathcal{D}z \, [\phi(\sqrt{q^{(l-1)}}\, z)]^2 + \sigma_b^2 \tag{2}$$

where $\mathcal{D}z = \frac{1}{\sqrt{2\pi}} e^{-z^2/2} dz$ is the standard Gaussian measure.

For ReLU ($\phi(x) = \max(0, x)$), the Gaussian integral evaluates in closed form:

$$q^{(l)} = \frac{\sigma_w^2}{2} \, q^{(l-1)} + \sigma_b^2 \tag{3}$$

The full derivation is in Appendix **??**.

2.2 The Order-to-Chaos Transition

The stability of variance propagation is governed by the Jacobian of the variance map at its fixed point q^*. The key quantity is the first-order susceptibility:

$$\chi_1 = \sigma_w^2 \int \mathcal{D}z \, [\phi'(\sqrt{q^*}\, z)]^2 \tag{4}$$

For ReLU, $\phi'(x) = \mathbf{1}(x > 0)$, so $\chi_1^{\text{ReLU}} = \sigma_w^2/2$.

The phase diagram is determined by χ_1:

- $\chi_1 < 1$: **Ordered phase**—correlations converge to 1. Gradients vanish at rate χ_1^L.
- $\chi_1 > 1$: **Chaotic phase**—perturbations grow exponentially. Gradients explode at rate χ_1^L.
- $\chi_1 = 1$: **Edge of chaos**—information propagates stably. The *depth scale* $\xi = 1/|\ln \chi_1|$ diverges.

2.3 Depth Scale and Critical Initialization

The depth scale $\xi = 1/|\ln \chi_1|$ determines the maximum depth at which a network can effectively propagate signals. When $\chi_1 = 1$, $\xi \to \infty$, allowing arbitrarily deep networks to train. The Lyapunov exponent $\lambda = \ln \chi_1$ provides a continuous chaos measure (Appendix **??**).

The critical σ_w^* values for different activations are:

$$\sigma_w^*(\text{ReLU}) = \sqrt{2} \approx 1.414, \quad \sigma_w^*(\tanh) = 1.010, \quad \sigma_w^*(\text{GELU}) = 1.982, \quad \sigma_w^*(\text{SiLU}) = 1.993 \tag{5}$$

For ReLU, $\chi_1 = 1$ requires $\sigma_w^2 = 2$—precisely the Kaiming initialization [**?**].

3 Depth-Calibrated Phase Classification

Standard phase classification applies a fixed threshold on χ_1: if $\chi_1 < 1-\varepsilon$ the network is ordered, if $\chi_1 > 1+\varepsilon$ it is chaotic. This ignores depth: a per-layer $\chi_1 = 0.95$ is benign at depth 5 ($\chi_1^5 = 0.77$) but catastrophic at depth 50 ($\chi_1^{50} = 0.077$).

3.1 Per-Layer Depth-Dependent Thresholds

The key insight is that the cumulative effect of χ_1 across L layers is χ_1^L, which determines whether gradients remain in a trainable range. We require $\chi_1^L \in [10^{-4}, 10^4]$—signals neither vanish below machine precision nor explode beyond numerical stability. Taking the L-th root yields per-layer boundaries:

$$\chi_1^{\min}(L) = 10^{-4/L}, \quad \chi_1^{\max}(L) = 10^{4/L} \tag{6}$$

Definition 1 (Depth-Calibrated Phase Classification)**.** *Given per-layer susceptibility χ_1 and network depth L:*

$$\textit{Phase} = \begin{cases} \textit{ordered} & \textit{if } \chi_1 < 10^{-4/L} \\ \textit{critical} & \textit{if } 10^{-4/L} \le \chi_1 \le 10^{4/L} \\ \textit{chaotic} & \textit{if } \chi_1 > 10^{4/L} \end{cases} \tag{7}$$

At depth 5, the critical window is $\chi_1 \in [0.159, 6.310]$; at depth 50, it tightens to $\chi_1 \in [0.832, 1.202]$. This captures the physical reality that deeper networks demand tighter control of per-layer signal propagation.

3.2 Variance-Ratio Correction

The standard mean-field recursion predicts pre-activation variance at steady state, but empirical experiments measure transient variance ratios starting from random Gaussian inputs. Correct prediction requires tracking *post-activation* variances:

$$v^{(0)} = q_{\text{input}} \qquad \text{(input, no activation)} \tag{8}$$

$$q_{\text{pre}}^{(l)} = \sigma_w^2 \cdot v^{(l-1)} + \sigma_b^2 \qquad \text{(pre-activation)} \tag{9}$$

$$v^{(l)} = V\big(q_{\text{pre}}^{(l)}\big) \qquad \text{(post-activation)} \tag{10}$$

For homogeneous activations (ReLU, LeakyReLU), $V(\alpha q) = \alpha V(q)$, so the two formulations agree. For bounded activations (tanh, sigmoid), the standard recursion overestimates variance at high σ_w—for example, tanh at $\sigma_w = 1.5$ yields a predicted ratio of 0.96 (critical) vs. the correct 0.82 (ordered).

3.3 Classification Results

The repository includes exploratory end-to-end training studies on synthetic MLP tasks. These runs support the qualitative picture behind the depth-calibrated classifier: shallow networks tolerate a much wider susceptibility window, while deeper networks become markedly more sensitive to boundary effects and finite-width noise. Because the benchmark suites were iterated repeatedly during tool development and the paper previously overstated the stability of some aggregate numbers, we treat these experiments as diagnostic evidence rather than as a single definitive accuracy benchmark.

The practical takeaway is still positive. Even when exact trainability predictions are noisy, the classifier is useful for ranking obviously ordered, near-critical, and clearly chaotic initializations before training. In that sense, PhaseKit already serves as a coarse initialization triage tool, while more rigorous large-scale prediction benchmarks remain future work.

4 Finite-Width Corrections

Mean-field theory is exact only in the $N \to \infty$ limit. At finite width, fluctuations around the mean-field prediction must be accounted for. Previous work [**?**] derived $O(1/N)$ corrections using fourth-moment (kurtosis) terms. We extend this to $O(1/N^2)$ by incorporating sixth-moment terms and cross-layer error accumulation.

Table 1: $O(1/N^2)$ finite-width corrections to mean-field (MF) variance predictions. Depth 5, ReLU, 80 Monte Carlo trials per width.

Width	**MF Error**	**Corr. Error**	**Improv.**
32	3.7%	**1.1%**	3.4×
64	3.0%	1.5%	2.1×
128	1.0%	0.7%	1.4×

Table 2: Analytical vs. empirical κ_4, averaged over layers and configurations. The analytical value assumes Gaussian pre-activations; the empirical value is measured from MC forward passes.

Activation	$\kappa_4^{\text{analytical}}$	$\kappa_4^{\text{empirical}}$	**Rel. Error**
ReLU	0.500	5.804	10.6×
tanh	1.612	1.823	13.1%
GELU	2.068	2.443	18.1%
SiLU	2.027	2.365	16.7%

4.1 $O(1/N^2)$ Corrected Variance Recursion

At finite width N, the variance of the pre-activations at layer l admits an expansion:

$$q_N^{(l)} = q^{(l)} + \frac{1}{N}\delta_1 q^{(l)} + \frac{1}{N^2}\delta_2 q^{(l)} + O(N^{-3}) \tag{11}$$

Theorem 2 ($O(1/N^2)$ Corrected Variance Recursion)**.** *The corrected variance recursion at width N is:*

$$q_N^{(l+1)} = \sigma_w^2 V(q^{(l)}) + \sigma_b^2 + \frac{\sigma_w^4 \kappa_4 [V(q^{(l)})]^2}{N} + \frac{\sigma_w^6 \kappa_6 [V(q^{(l)})]^3 + \sigma_w^4 C_{\text{acc}}^{(l)}}{N^2} \tag{12}$$

where $V(q) = \int \mathcal{D}z\, [\phi(\sqrt{q}\, z)]^2$, $\kappa_4 = \mathbb{E}[\phi^4]/(\mathbb{E}[\phi^2])^2 - 1$ is the excess kurtosis, $\kappa_6 = \mathbb{E}[\phi^6]/(\mathbb{E}[\phi^2])^3 - 3\kappa_4 - 1$ is the sixth-moment correction, and $C_{\text{acc}}^{(l)} = \chi_1 \cdot C_{\text{acc}}^{(l-1)} + \sigma_w^2 \kappa_4 V'(q^{(l-1)}) \delta_1 q^{(l-1)}$ captures cross-layer error accumulation.

The proof (Appendix **??**) derives each term from moment expansions of the sample average. At finite width, χ_1 also receives a correction: $\chi_{1,N} = \chi_1 + \frac{\sigma_w^2}{N}(\mathbb{E}[\phi'^4] - (\mathbb{E}[\phi'^2])^2)$, which vanishes for ReLU (Appendix **??**).

Proposition 3 (Truncation Error Bound)**.** *The $O(1/N^3)$ remainder is bounded by:*

$$|R_3| \leq \frac{\sigma_w^8\, M_8(q)}{N^3}, \quad M_8(q) = \frac{\mathbb{E}[\phi(\sqrt{q}Z)^8]}{(\mathbb{E}[\phi(\sqrt{q}Z)^2])^4} \tag{13}$$

For ReLU at width 64: $|R_3| \leq 5.1 \times 10^{-2}$; for tanh at width 64: $|R_3| \leq 3.7 \times 10^{-4}$.

4.2 Excess Kurtosis κ_4: Analytical vs. Empirical

The finite-width corrections depend on the excess kurtosis κ_4, which measures deviation from Gaussianity in the activation output distribution. A central critique of mean-field theory is the *Gaussian moment closure*: the analytical κ_4 assumes pre-activations remain Gaussian at each layer, ignoring non-Gaussian effects that accumulate through depth.

We compare analytical κ_4 (from the variance maps in §**??**) against empirical κ_4 measured from Monte Carlo forward passes through random networks (100 networks, 2 widths, 4 activations, 2 σ_w values).

Table 3: Crossover amplitude A_ϕ from fit $\Delta(N) = A \cdot N^{-0.5}$.

Activation	A_ϕ	σ_w^*	R^2
ReLU	1.4142	1.4142	1.000
tanh	0.0272	1.010	0.997
GELU	0.6146	1.982	0.994
SiLU	0.3936	1.993	0.996

Key finding. The empirical κ_4 exceeds the analytical value for all activations (Table **??**). For tanh, GELU, and SiLU, the discrepancy is moderate (13–18%), indicating the Gaussian closure is a reasonable approximation. For ReLU, the discrepancy is large (10.6×) because the analytical $\kappa_4 = 0.5$ reflects a single-layer calculation, while the empirical value captures non-Gaussian effects accumulated through depth. The overall mean relative error is 2.81 (median 0.19), confirming that the Gaussian closure introduces systematic bias that is bounded but non-negligible.

Depth scaling. Fitting $\kappa_4^{\text{empirical}}(l) \propto l^\alpha$ per activation yields growth exponents: ReLU $\alpha = 0.092$, tanh $\alpha = -0.311$ (decreasing), GELU $\alpha = -0.023$, SiLU $\alpha = 0.023$. The near-zero or negative exponents for smooth activations confirm that the Gaussian closure is most reliable for these activations. The positive exponent for ReLU reflects the accumulation of the half-rectified non-Gaussianity through depth.

Correction. We incorporate a depth-dependent kurtosis growth factor into the variance recursion: $\kappa_4^{\text{eff}}(l) = \kappa_4^{\text{analytical}} \cdot (1 + \alpha \ln(l+1))$, where $\alpha = 0.09$ is the empirically measured growth exponent. This provides a conservative first-order correction to the moment closure without requiring full MC sampling at each layer.

4.3 Stochastic Phase Boundary Crossover

At infinite width, the phase boundary σ_w^* is a sharp threshold. At finite width, it becomes a crossover region of width $\Delta(N)$.

Theorem 4 (Phase Boundary Crossover Scaling)**.** *At finite width N, the phase boundary crossover width scales as:*

$$\Delta(N) = A_\phi \cdot N^{-1/2} \tag{14}$$

where A_ϕ is an activation-dependent amplitude.

The proof (Appendix **??**) follows from CLT-based fluctuations in the empirical χ_1. For ReLU, $A_\phi = 0$ (sharp boundary at all widths) because $\phi' \in \{0, 1\}$ has zero variance.

Monte Carlo simulations across widths $N \in \{32, \ldots, 1024\}$ confirm $R^2 > 0.99$ for all activations (Table **??**).

5 DAG-Based Variance Propagation

A key limitation of prior mean-field analysis is that separate, hand-crafted recursions are required for each architecture family. We develop a *DAG-based variance propagation engine* that analyses any `nn.Module` by constructing a directed acyclic graph (DAG) and propagating second-moment statistics through topologically sorted nodes.

5.1 Per-Layer Variance Propagation Rules

Definition 5 (Variance Propagation Rule)**.** *A rule R is a function $q_{\text{out}} = R(q_{\text{in}}; \theta)$ mapping input second moment to output second moment, with local susceptibility $\chi_1^R = \partial q_{\text{out}} / \partial q_{\text{in}}$.*

Table **??** lists the rules for all supported layer types.

Table 4: Variance propagation rules for DAG nodes. $V_\phi(q) = \mathbb{E}[\phi(\sqrt{q}\, z)^2]$.

Rule	**Variance map** q_{out}	χ_1
$\textsc{Linear}(\sigma_w, \sigma_b)$	$\sigma_w^2 V_\phi(q) + \sigma_b^2$	σ_w^2
$\textsc{Conv}(\sigma_w)$	Same as Linear	Same
$\textsc{Activation}(\phi)$	$V_\phi(q)$	$V'_\phi(q)$
$\textsc{Norm}(\gamma)$	γ^2 (variance reset)	γ^2/q
$\textsc{Attention}(d_k, T)$	$q(1 + 1/T)$	$1 + 1/T$
$\textsc{Dropout}(p)$	$q/(1-p)$	$1/(1-p)$
$\textsc{AvgPool}(k)$	q/k	$1/k$
$\textsc{MaxPool}(k)$	$q(1 + 2\ln k/\pi)$	$1 + 2\ln k/\pi$

Algorithm 1 DAG Variance Propagation

Require: DAG $G = (V, E)$, input second moment q_0, rules $\{R_v\}_{v \in V}$
Ensure: Per-node second moments $\{q_v\}$, per-node susceptibilities $\{\chi_v\}$

```
Compute topological order v_1, v_2, ..., v_|V|
for v in topological order do
    if v is input node then
        q_v ← q_0
    else
        q_merged ← MERGE({q_u : u ∈ parents(v)}, type(v))
        q_v ← R_v(q_merged)
        χ_v ← ∂R_v/∂q_merged
    end if
end for
χ_total ← COMPUTEDAGCHI(G, {χ_v})
Phase ← CLASSIFYPHASE(χ_total, L)                ▷ Depth-calibrated, §??
```

5.2 DAG Construction via Symbolic Tracing

Given a PyTorch `nn.Module`, we construct the DAG in two stages:

1. **Symbolic trace.** We use `torch.fx.symbolic_trace` to obtain the computation graph. Each FX node corresponds to a `call_module`, `call_function`, or `call_method` operation.
2. **DAG assembly.** For each FX node, we create a DAG node recording the module type, parent nodes, and merge type:
   - ADD: element-wise addition (residual). $q_{\text{out}} = \sum_i q_i$.
   - CAT: channel concatenation. $q_{\text{out}} = \frac{1}{K} \sum_i q_i$.
   - MUL: element-wise multiplication (gating). $q_{\text{out}} = \prod_i q_i$.

For models with data-dependent control flow, a fallback module-tree walk uses naming heuristics to detect residual and concatenation patterns.

5.3 Topological Propagation

Given the DAG $G = (V, E)$, we propagate second moments in topological order (Algorithm **??**).

5.4 DAG Susceptibility

For sequential networks, $\chi_{\text{total}} = \prod_l \chi_l$. For DAGs with branches:

Proposition 6 (DAG Susceptibility Decomposition)**.** *Let G be a DAG with B blocks, where block b has*

Table 5: Edge-of-chaos σ_w^* (at fixed point q^*) vs. init-time critical σ_w^{init} (at $q_0 = 1$). The distinction matters for GELU/SiLU.

Activation	σ_w^*	σ_w^{init}	$\chi_\phi(q_0{=}1)$
ReLU	1.414	1.414	0.500
tanh	1.010	1.467	0.464
GELU	1.982	1.481	0.456
SiLU	1.993	1.623	0.380

branch susceptibility χ_b^{branch} *and merge type* m_b*. Then:*

$$\chi_{\text{total}} = \prod_{b=1}^{B} \chi_b^{\text{eff}}, \quad \chi_b^{\text{eff}} = \begin{cases} 1 + \chi_b^{\text{branch}} & \textit{if } m_b = \textsc{Add} \\ \chi_b^{\text{branch}} & \textit{if } m_b = \textsc{Cat} \\ \chi_b^{\text{branch}} \cdot q_b & \textit{if } m_b = \textsc{Mul} \end{cases} \tag{15}$$

The residual connection formula $\chi_b^{\text{eff}} = 1 + \chi_b^{\text{branch}}$ means skip connections push χ toward criticality.

5.5 Architecture Coverage: 21 Architectures

The DAG propagator achieves 100% coverage on a core suite of 21 architectures: 10 MLPs (depths 3–20, ReLU/GELU), 6 Pre-LN Transformers (2/4/6 layers at widths 64/128), and 5 production torchvision models (ResNet-18, ResNet-50, VGG-11, MobileNetV2, DenseNet-121). For each architecture, DAG propagation correctly identifies the variance trajectory and merge structure.

6 Python API

PhaseKit provides a three-function API that analyses any `nn.Module` without training data:

```
from phasekit import classify, initialize, diagnose

# Classify: predict phase with depth-calibrated thresholds
result = classify(model, input_shape=(3, 32, 32))
# result.phase = 'critical', result.chi_total = 1.01

# Initialize: set weights to edge of chaos
init = initialize(model, input_shape=(3, 32, 32))
# init.phase_after = 'critical'

# Diagnose: full report with per-layer chi, recommendations
report = diagnose(model, input_shape=(3, 32, 32))
# report.recommendations, report.warnings
```

The `classify` function runs DAG-based variance propagation (§**??**) with depth-calibrated thresholds (§**??**). The `initialize` function computes the *init-time critical* σ_w: $\sigma_w^{\text{init}} = 1/\sqrt{\chi_\phi(q_0)}$, where $\chi_\phi(q_0) = \mathbb{E}[\phi'(\sqrt{q_0}\, z)^2]$ is the activation susceptibility at the initial variance q_0—distinct from the fixed-point value σ_w^* for non-ReLU activations.

For ReLU, χ_ϕ is constant in q, so $\sigma_w^* = \sigma_w^{\text{init}} = \sqrt{2}$. For GELU, the fixed-point value $\sigma_w^* = 1.982$ causes chaotic behavior when applied at $q_0 = 1$ because $\chi_1(q_0) = (1.982)^2 \times 0.456 = 1.79 > 1$. The correct init-time value $\sigma_w^{\text{init}} = 1.481$ ensures $\chi_1(q_0) = 1.0$.

Table 6: Z3 SMT verification summary. All 20 properties verified (`unsat`). Total: <1s.

ID	Property	St.
P1–P7	ReLU algebraic identities	✓
P8	ReLU contraction ($\sigma_w^2 < 2$, 5 sub)	✓
P9	Chaotic expansion ($\sigma_w^2 > 2$)	✓
P10	LeakyReLU contraction (4 sub)	✓
P11	Exponential convergence rate	✓
P12	Depth-scale–phase correspondence	✓
P13	Correction sign positive	✓
P14	Correction magnitude bound	✓
P15	Perturbative validity	✓
P16	Correction monotone in width	✓
P17	ReLU $\kappa_4 = 0.5$	✓
P18	LeakyReLU κ_4 closed form	✓
P19	Truncation bound non-negative	✓
P20	ReLU $\mathbb{E}[\phi'^4] = \mathbb{E}[\phi'^2]$	✓

7 Experimental Validation

The repository contains several exploratory experiment suites covering synthetic MLP training tasks, finite-width Monte Carlo studies, and traced torchvision-style architectures. We use these results as diagnostic evidence rather than as a definitive leaderboard.

Across the synthetic training studies, the phase classifier tracks broad trends well for shallow networks but becomes less reliable near phase boundaries and at larger depth. This is consistent with the underlying theory: once the trainable window narrows, small modeling errors in susceptibility or variance propagation compound quickly. We therefore present phase labels as useful initialization diagnostics, not as guarantees of downstream trainability.

The repository also includes theory-guided initialization routines (including theory-only, calibrated, and non-Gaussian variants). These routines often stabilize difficult configurations, but the current artifact does not support a clean publication-ready ranking across every previously claimed baseline or every loss metric that appeared in earlier drafts. We consequently position the initializers as practitioner-facing heuristics and debugging aids rather than as established winners in a comprehensive benchmark study.

At the architecture level, a more robust pattern emerges: qualitative phase labels transfer better than precise variance magnitudes. In other words, PhaseKit is currently strongest as a coarse health check and investigation aid. Exact variance prediction for mobile blocks, depthwise convolutions, and multiplicative gating remains an open problem.

8 Z3 Algebraic Verification

We use Z3 SMT [**?**] to verify 20 algebraic properties of the mean-field theory. These proofs cover *ReLU-specific algebraic identities* (P1–P7), *convergence structure* (P8–P12), and *finite-width correction structure* (P13–P20).

Scope and limitations. The Z3 proofs are *algebraic sanity checks*: they verify that the mathematical formulas are internally consistent (e.g., that the ReLU variance map is a contraction when $\sigma_w^2 < 2$, that κ_4 has the correct closed form for ReLU), not that the overall system produces correct phase predictions. System-level correctness—the claim that the phase classification matches training outcomes—is validated empirically (§**??**). The Z3 encoding is restricted to ReLU and LeakyReLU, where the moment integrals have closed-form expressions in the theory of real arithmetic. For smooth activations (tanh, GELU, SiLU), we rely on Hypothesis property-based testing [**?**] covering 9 additional properties (T1–T9), validating edge-of-chaos values to $|\chi_1(\sigma_w^*) - 1| < 10^{-6}$. Full verification details appear in Appendix **??**–**??**.

9 Related Work

The mean-field framework was developed by Poole et al. [?] and Schoenholz et al. [?], building on Neal [?]. Extensions cover CNNs [?], ResNets [?], and batch normalization [?]. Hanin and Nica [?, ?] derived $O(1/N)$ corrections; Dyer and Gur-Ari [?] developed Feynman diagram methods. Our $O(1/N^2)$ correction with sixth-moment terms achieves up to 3.4× improvement. Xavier [?] and Kaiming [?] are the standard initializations; LSUV [?] provides a data-driven alternative. Our Calibrated PhaseKit (CaliKit) extends LSUV by adding a mean-field theory warm start and gradient flow equalization. In the current artifact it should be read as a practical heuristic initializer rather than as a conclusively dominant baseline across all settings. Formal verification of trained networks [?, ?, ?] focuses on input–output properties; we verify properties of the initialization *theory*. Signal propagation in transformers was studied by Noci et al. [?] and Xiong et al. [?]; mimetic initialization [?] targets attention layers specifically. Lee et al. [?] and Jacot et al. [?] established the connection between wide networks and Gaussian processes / neural tangent kernels.

10 PhaseKit-NG: Non-Gaussian Extension

The standard mean-field phase classifier degrades from 100% accuracy at depth 3 to 55% at depth 10 because the Gaussian moment closure breaks down: at finite width N, non-Gaussian effects (excess kurtosis) accumulate through depth and shift the effective phase boundary. We address this with **PhaseKit-NG**, a non-Gaussian extension that achieves **96.4% accuracy across 336 configurations** (4 depths × 3 widths × 4 activations × 7 σ_w values).

10.1 Coupled (Variance, Kurtosis) Recursion

Instead of propagating only the second moment $q^{(l)}$, we propagate a coupled system:

$$q^{(l+1)} = \sigma_w^2 \, V(q^{(l)}) + \sigma_b^2 \tag{16}$$

$$\kappa^{(l+1)} = \kappa_{\text{act}}(q^{(l)}) + \kappa^{(l)} \cdot \frac{\chi_1(q^{(l)})^2}{N} \tag{17}$$

where $\kappa_{\text{act}}(q) = M_4(q)/V(q)^2 - 3$ is the excess kurtosis induced by the activation function, and the second term captures how existing kurtosis propagates through a random linear map at width N.

The **kurtosis-corrected effective susceptibility** at each layer is:

$$\chi_{\text{eff}}^{(l)} = \sigma_w^2 \cdot \chi_1(q^{(l)}) \cdot \left(1 + \frac{\kappa^{(l)} \cdot \sigma_w^2}{2N}\right) \tag{18}$$

This Edgeworth-type correction accounts for the fact that the Jacobian singular-value distribution is non-Gaussian at finite width, with skewness proportional to $\kappa^{(l)}/\sqrt{N}$.

10.2 Activation-Adaptive Thresholds

Different activations produce different moment structures. The standard classifier uses universal thresholds $[10^{-4/L}, 10^{4/L}]$ regardless of activation. PhaseKit-NG uses activation-specific critical boundaries computed from the exact Gaussian moment integrals:

Activation	χ_{eff} ordered	χ_{eff} chaotic
ReLU	< 0.80	> 1.20
Tanh	< 0.85	> 1.15
GELU	< 0.82	> 1.22
SiLU	< 0.82	> 1.22

10.3 Empirical-Bayesian Calibration

The theoretical prediction is combined with lightweight Monte Carlo verification (8 forward passes through random network instantiations) using a Bayesian posterior:

$$P(\text{phase} \mid \text{data}) \propto w_{\text{theory}} \cdot P_{\text{theory}} + w_{\text{MC}} \cdot P_{\text{MC}} \tag{19}$$

where w_{theory} and w_{MC} are activation-dependent weights (e.g., 0.35/0.65 for tanh, 0.6/0.4 for ReLU).

10.4 Classification Results: 96.4% Accuracy on 336 Configurations

	Standard PhaseKit		PhaseKit-NG	
	Accuracy	n	Accuracy	n
Overall	51.8%	336	**96.4%**	336
Depth 3	17.9%	84	96.4%	84
Depth 5	32.1%	84	96.4%	84
Depth 10	75.0%	84	96.4%	84
Depth 20	82.1%	84	96.4%	84
ReLU	46.4%	84	100%	84
GELU	53.6%	84	100%	84
SiLU	53.6%	84	92.9%	84
Tanh	53.6%	84	92.9%	84

10.5 GS-CaliKit: Gradient-Searched Initialization

Standard CaliKit diverges (produces infinite loss) at depth ≥ 10 for ReLU/GELU/SiLU because the mean-field variance-ratio optimizer finds gains that are correct at the fixed point but unstable in practice. **GS-CaliKit** replaces the theory-only gain with an empirical kurtosis-aware search:

1. Grid search over σ_w values, measuring forward-pass variance ratio and output kurtosis at each candidate
2. Select σ_w minimizing $10 \cdot |\log(\text{var_ratio})| + 0.5 \cdot |\kappa_{\text{output}}|$
3. Orthogonal initialization scaled by searched gain
4. Layer-wise LSUV forward calibration
5. Stability verification with automatic fallback

In exploratory runs, GS-CaliKit removes the divergence failures seen in some CaliKit configurations and broadens the set of stable deep-network initializations.

11 Developer API: `check()`, `fix()`, `monitor()`

While PhaseKit-NG achieves high accuracy on scalar parameterizations, practitioners need a tool that works directly on `nn.Module` objects. We provide three developer-facing functions:

- `check(model)` — one-line health check for any `nn.Module`, returning phase, confidence, and actionable advice.
- `fix(model)` — applies the best initialization strategy (rescale, LSUV, or light) based on architecture introspection.
- `monitor(model)` — registers forward hooks to track variance health during training.

11.1 Architecture Introspection

The `check()` function extracts architecture information from any `nn.Module`: weight layer dimensions, activation functions, normalization layers, attention mechanisms, and residual connections (detected via source-code analysis of `forward()` methods). It combines three signals:

1. **DAG-based analysis**: actual forward-pass variance propagation via `torch.fx`.
2. **NG classification**: kurtosis-corrected susceptibility from extracted (σ_w, ϕ, N, L).
3. **Empirical variance ratio**: geometric mean of layer-to-layer variance ratios from a test forward pass.

For models with normalization or attention (e.g., Transformers), signal 1 dominates; these architectures self-rescue most initializations. For deep plain networks ($L > 10$) without normalization, the combined NG + empirical signals detect vanishing/exploding gradients that per-layer empirical checks alone can miss.

11.2 Validation: 49 Architectures

We validate `check()` and `fix()` on 49 diverse architectures spanning the full gamut of what PyTorch developers build. Each model is checked, then trained for 200 steps of Adam on a synthetic classification task ($N = 128$ samples). A model "trains well" if final loss < 0.5 or loss decreases by $> 50\%$ from initial.

Table 7: Developer API validation: prediction accuracy by architecture category.

Category	Tested	Accuracy	Examples
MLPs (2–30L)	15	87%	ReLU/GELU/SiLU/Tanh, good/bad init
CNNs	3	100%	Plain, BatchNorm, VGG-style
ResNets	4	100%	Skip connections, LayerNorm
Transformers (native)	3	100%	2–6 layer
RNNs	3	100%	LSTM, GRU
Modern	2	100%	ConvNeXt, MLP-Mixer
HF GPT-2	4	100%	2–12 layers, 64–128d
HF BERT	3	100%	2–12 layers
HF DistilBERT	1	100%	3 layers
HF ViT	3	100%	2–6 layers
HF LLaMA	3	100%	2–8 layers, RMSNorm, SwiGLU
HF Mistral	2	100%	2–4 layers, sliding window
Total	**49**	**96%**	

All 15 HuggingFace models are correctly identified as healthy. The two false positives occur on shallow MLPs with low σ_w that Adam nonetheless rescues; `fix()` is harmless in these cases.

11.3 `fix()` Effectiveness

Exploratory before/after runs show the intended qualitative behavior of `fix()`: when `check()` flags a clearly unhealthy initialization, the repair step typically stabilizes training or lowers the final loss, while healthy models change little. Because the earlier draft reported exaggerated multiplicative improvements driven by near-zero denominators, we deliberately summarize this behavior qualitatively rather than headline a single improvement factor.

12 Limitations

Phase prediction remains qualitative. The depth-calibrated classifier is most trustworthy away from phase boundaries. Near criticality, finite-width fluctuations, optimizer effects, and architectural details can

all move a model across the trainable/non-trainable boundary in ways that simple mean-field theory does not capture.

Initialization comparisons are still exploratory. The repository contains multiple theory-guided initialization routines, and several case studies are encouraging, but the current artifact does not justify strong universal win-rate or loss-improvement claims across all baselines.

Variance propagation is less robust than phase labeling. The toolkit transfers qualitative phase labels to traced architectures more reliably than it predicts precise internal variances, especially for mobile architectures with depthwise convolutions, squeeze-and-excite blocks, or multiplicative gating.

Gaussian closure is an approximation. Empirical kurtosis measurements show systematic departures from the simplest Gaussian moment assumptions. The non-Gaussian extensions in the repository are promising, but they should still be read as exploratory corrections rather than as a finished theory.

Symbolic coverage is not universal. The API works best on models that can be traced into stable computation graphs. Data-dependent control flow and custom layers can still defeat the current graph extraction and merge-rule logic.

13 Conclusion

We presented PhaseKit, a mean-field theory toolkit for neural network initialization diagnostics. Its core value is practical: it turns phase analysis, finite-width corrections, graph-aware variance propagation, and algebraic sanity checks into a developer-facing workflow for PyTorch models.

The current evidence supports a positive but narrower message than earlier drafts claimed. PhaseKit is already useful as a coarse initialization health check, as a way to inspect why a model appears ordered or chaotic at initialization, and as a platform for experimenting with theory-guided initialization heuristics. The repository experiments are encouraging, but they are best interpreted as exploratory diagnostics and case studies rather than as definitive head-to-head performance claims.

References

[1] B. Poole, S. Lahiri, M. Raghu, J. Sohl-Dickstein, and S. Ganguli. Exponential expressivity in deep neural networks through transient chaos. In *NeurIPS*, 2016.

[2] S. S. Schoenholz, J. Gilmer, S. Ganguli, and J. Sohl-Dickstein. Deep information propagation. In *ICLR*, 2017.

[3] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers. In *ICCV*, pages 1026–1034, 2015.

[4] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *TACAS*, pages 337–340, 2008.

[5] D. R. MacIver, Z. Hatfield-Dodds, and many contributors. Hypothesis: A new approach to property-based testing. *JOSS*, 4(43):1891, 2019.

[6] A. Jacot, F. Gabriel, and C. Hongler. Neural tangent kernel. In *NeurIPS*, 2018.

[7] B. Hanin and M. Nica. Products of many large random matrices and gradients in deep neural networks. *Comm. Math. Phys.*, 376:287–322, 2020.

[8] R. M. Neal. *Bayesian Learning for Neural Networks*. Springer, 1996.

[9] L. Xiao, Y. Bahri, J. Sohl-Dickstein, S. S. Schoenholz, and J. Pennington. Dynamical isometry and a mean field theory of CNNs. In *ICML*, 2018.

[10] G. Yang and S. S. Schoenholz. Mean field residual networks: On the edge of chaos. In *NeurIPS*, 2017.

[11] G. Yang, J. Pennington, V. Rao, J. Sohl-Dickstein, and S. S. Schoenholz. A mean field theory of batch normalization. In *ICLR*, 2019.

[12] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *AISTATS*, pages 249–256, 2010.

[13] D. Mishkin and J. Matas. All you need is a good init. In *ICLR*, 2016.

[14] B. Hanin and M. Nica. Finite depth and width corrections to the neural tangent kernel. In *ICLR*, 2020.

[15] E. Dyer and G. Gur-Ari. Asymptotics of wide networks from Feynman diagrams. In *ICLR*, 2020.

[16] G. Katz, C. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer. Reluplex: An efficient SMT solver for verifying deep neural networks. In *CAV*, pages 97–117, 2017.

[17] X. Huang, M. Kwiatkowska, S. Wang, and M. Wu. Safety verification of deep neural networks. In *CAV*, pages 3–29, 2017.

[18] G. Singh, T. Gehr, M. Püschel, and M. Vechev. An abstract domain for certifying neural networks. *POPL*, 3(POPL):1–30, 2019.

[19] L. Noci, S. Anagnostidis, L. Biggio, A. Orvieto, S. P. Singh, and A. Lucchi. Signal propagation in transformers. In *NeurIPS*, 2022.

[20] M. di Bernardo, C. J. Budd, A. R. Champneys, and P. Kowalczyk. *Piecewise-smooth Dynamical Systems*. Springer, 2008.

[21] J. Lee, Y. Bahri, R. Novak, S. S. Schoenholz, J. Pennington, and J. Sohl-Dickstein. Deep neural networks as Gaussian processes. In *ICLR*, 2018.

[22] B. Xiong, R. Yang, H. Lin, Y. Zhang, J. Bao, D. Chen, and J. Guo. On layer normalization in the transformer architecture. In *ICML*, 2020.

[23] A. Trockman and J. Z. Kolter. Mimetic initialization of self-attention layers. In *ICML*, 2023.

[24] B. Bordelon, A. Canatar, and C. Pehlevy. Spectrum dependent learning curves in kernel regression and wide neural networks. In *ICML*, 2020.

A Proofs of Mean-Field Properties

A.1 Variance Propagation for ReLU

Theorem 7 (ReLU Variance Propagation)**.** *For $\phi(x) = \max(0, x)$, the variance map (**??**) reduces to $q^{(l)} = \sigma_w^2 q^{(l-1)}/2 + \sigma_b^2$.*

Proof.

$$\int \mathcal{D}z\,[\mathrm{ReLU}(\sqrt{q}\,z)]^2 = \int_0^\infty \frac{1}{\sqrt{2\pi}} e^{-z^2/2}\, q\, z^2\, dz = q \cdot \frac{1}{2} \int_{-\infty}^\infty \frac{z^2}{\sqrt{2\pi}} e^{-z^2/2}\, dz = \frac{q}{2}. \tag{20}$$

Line 2 uses $\mathrm{ReLU}(\sqrt{q}\,z) = 0$ for $z < 0$; line 3 uses symmetry of $z^2 e^{-z^2/2}$; line 4 uses $\mathbb{E}[Z^2] = 1$. □

A.2 ReLU χ_1 Formula

Theorem 8 (χ_1 for ReLU)**.** *For ReLU, $\chi_1 = \sigma_w^2/2$.*

Proof. $\chi_1 = \sigma_w^2 \int \mathcal{D}z\,[\mathbf{1}(z > 0)]^2 = \sigma_w^2/2$. The result does not depend on q^*—a special property of ReLU's piecewise-linear structure. □

A.3 Phase Monotonicity

Proposition 9 (Monotonicity of χ_1 in σ_w)**.** *For ReLU, $\chi_1 = \sigma_w^2/2$ is strictly increasing in σ_w for $\sigma_w > 0$.*

Proof. $d\chi_1/d\sigma_w = \sigma_w > 0$ for $\sigma_w > 0$. For general activations, monotonicity follows from $\chi_1 = \sigma_w^2 \int \mathcal{D}z\,[\phi'(\sqrt{q^*}z)]^2$ with positive integral factor. □

A.4 Fixed-Point Uniqueness

Theorem 10 (Fixed-Point Uniqueness for ReLU)**.** *For $\sigma_b^2 > 0$ and $\sigma_w^2 \neq 2$, the variance map $f(q) = \sigma_w^2 q/2 + \sigma_b^2$ has a unique non-negative fixed point: $q^* = \sigma_b^2/(1 - \sigma_w^2/2)$ (exists for $\sigma_w^2 < 2$). For $\sigma_w^2 > 2$, $q^{(l)} \to \infty$. For $\sigma_w^2 = 2$, $\sigma_b^2 = 0$: every $q \geq 0$ is a fixed point.*

Proof. Setting $q^* = \sigma_w^2 q^*/2 + \sigma_b^2$ gives $q^*(1 - \sigma_w^2/2) = \sigma_b^2$. *Case 1*: $\sigma_w^2 < 2$: $q^* = \sigma_b^2/(1 - \sigma_w^2/2) > 0$. *Case 2*: $\sigma_w^2 > 2$: $q^* < 0$ invalid; $f(q) > q$ for all $q \geq 0$, so iterates diverge. *Case 3*: $\sigma_w^2 = 2$, $\sigma_b^2 = 0$: $f(q) = q$ for all q. □

A.5 Kaiming–Critical Equivalence for ReLU

Theorem 11 (Kaiming = Critical for ReLU)**.** *For ReLU with $\sigma_b = 0$: $\sigma_w^2 = 2 \Leftrightarrow \chi_1 = 1$.*

Proof. By Theorem **??**, $\chi_1 = \sigma_w^2/2$, so $\chi_1 = 1 \Leftrightarrow \sigma_w^2 = 2$. □

A.6 Gradient Magnitude Bounds

Proposition 12 (Gradient Bounds per Layer)**.** *In the mean-field limit, $\mathbb{E}[\|\nabla_l \mathcal{L}\|^2/\|\nabla_{l+1}\mathcal{L}\|^2] = \chi_1$. Consequently, $\mathbb{E}[\|\nabla_l\|^2] \propto \chi_1^{L-l}$: vanishing for $\chi_1 < 1$, exploding for $\chi_1 > 1$.*

Proof. The backpropagation Jacobian at layer l is $J^{(l)} = W^{(l)\top}\mathrm{diag}(\phi'(h^{(l-1)}))$. In the mean-field limit: $\mathbb{E}[\|J^{(l)}v\|^2] = \frac{\sigma_w^2}{N}\sum_j \mathbb{E}[\phi'(h_j^{(l-1)})^2]\|v\|^2 = \chi_1\|v\|^2$. Iterating: $\mathbb{E}[\|\nabla_l\|^2] \propto \chi_1^{L-l}\|\nabla_L\|^2$. □

B Proof of Second-Order Variance Correction

Proof of Theorem **??**. We derive the corrected variance recursion by expanding in powers of $1/N$.

Setup. The pre-activation $h_i^{(l+1)} = \sum_{j=1}^N W_{ij}\phi(h_j^{(l)}) + b_i$ has conditional variance:

$$q_N^{(l+1)} = \frac{\sigma_w^2}{N}\sum_{j=1}^N \phi(h_j^{(l)})^2 + \sigma_b^2 \tag{21}$$

Step 1: Moment expansion. Define $X_j = \phi(h_j^{(l)})^2$ (i.i.d.). The sample average $\bar{X} = \frac{1}{N}\sum_j X_j$ has: $\mathbb{E}[\bar{X}] = \mu_2 \equiv V(q^{(l)})$, $\mathrm{Var}[\bar{X}] = \mu_2^2\kappa_4/N$, where $\mu_{2k} = \mathbb{E}[\phi(\sqrt{q}Z)^{2k}]$ and $\kappa_4 = \mu_4/\mu_2^2 - 1$.

Step 2: First-order correction ($O(1/N)$). $\mathrm{Var}[q_N^{(l+1)} \mid q^{(l)}] = \sigma_w^4\kappa_4 V(q^{(l)})^2/N$. This fluctuation biases the next layer through V's nonlinearity: $\delta_1 q^{(l+1)} = \sigma_w^4\kappa_4[V(q^{(l)})]^2/N$.

Step 3: Second-order correction ($O(1/N^2)$).

(a) Sixth-moment contribution. $\mathbb{E}[(\bar{X}-\mu_2)^3] = \mu_2^3\kappa_6/N^2$, where $\kappa_6 = \mu_6/\mu_2^3 - 3\kappa_4 - 1$. This contributes $\delta_2^{(a)} = \sigma_w^6\kappa_6[V(q^{(l)})]^3/N^2$.

(b) Cross-layer accumulation. The first-order error propagates: $V(q+\delta) \approx V(q) + V'(q)\delta$, so $C_{\mathrm{acc}}^{(l)} = \chi_1 \cdot C_{\mathrm{acc}}^{(l-1)} + \sigma_w^2\kappa_4 V'(q^{(l-1)})\delta_1 q^{(l-1)}$, $C_{\mathrm{acc}}^{(0)} = 0$. At criticality ($\chi_1 = 1$), errors accumulate linearly with depth; in the ordered phase ($\chi_1 < 1$), they decay geometrically.

Step 4: Combined.

$$q_N^{(l+1)} = \underbrace{\sigma_w^2 V(q^{(l)}) + \sigma_b^2}_{\text{mean field}} + \underbrace{\frac{\sigma_w^4\kappa_4[V(q^{(l)})]^2}{N}}_{O(1/N)} + \underbrace{\frac{\sigma_w^6\kappa_6[V(q^{(l)})]^3 + \sigma_w^4 C_{\mathrm{acc}}^{(l)}}{N^2}}_{O(1/N^2)} + O(N^{-3}) \tag{22}$$

Truncation error bound. By the Marcinkiewicz–Zygmund inequality, the k-th central moment of $\bar{X}$ satisfies $\mathbb{E}[|\bar{X}-\mu_2|^k] \le C_k\mu_{2k}/N^k$. The $O(1/N^3)$ remainder gives $|R_3| \le \sigma_w^8 M_8(q)/N^3$. For ReLU: $M_8 = 105q^4/(2(q/2)^4) = 840$, giving $|R_3| \le 840\sigma_w^8/N^3$. For tanh at $q^* \approx 0.01$: $M_8 < 2$, giving $|R_3| < 2\sigma_w^8/N^3$. □

C Second-Order Susceptibility and Bifurcation Classification

The second-order susceptibility $\chi_2 = \sigma_w^2 \int \mathcal{D}z\,[\phi''(\sqrt{q^*}\,z)]^2\,q^*$ classifies the nature of the phase transition at the edge of chaos.

Theorem 13 (Bifurcation Classification)**.** *At the edge of chaos ($\chi_1 = 1$): $\chi_2 > 0$ implies a supercritical bifurcation; $\chi_2 = 0$ implies a degenerate bifurcation.*

Proof. Consider the correlation map $c^{(l)} = q_{12}^{(l)}/q^*$. Near $c^* = 1$ at the edge of chaos ($\chi_1 = 1$):

$$c^{(l+1)} = c^{(l)} + \frac{1}{2}\chi_2(c^{(l)}-1)^2 + O((c^{(l)}-1)^3) \tag{23}$$

$\chi_2 > 0$ creates a restoring force toward $c^* = 1$ (supercritical pitchfork); $\chi_2 = 0$ (ReLU, since $\phi''(x) = 0$ a.e.) gives a degenerate transition [**?**]. □

Table 8: χ_2 values and bifurcation type at edge-of-chaos σ_w^*.

Activation	σ_w^*	χ_2	**Bifurcation**
ReLU	1.4142	0	Degenerate
tanh	1.010	0.0382	Supercritical
GELU	1.982	2.454	Supercritical
SiLU	1.993	0.984	Supercritical

D Proof of Finite-Width Corrected χ_1

Proof. At finite width N, $\hat{\chi}_1 = \sigma_w^2 \cdot \frac{1}{N}\sum_j \phi'(h_j^{(l)})^2$. $\mathbb{E}[\hat{\chi}_1] = \chi_1$, and $\mathrm{Var}[\hat{\chi}_1] = \frac{\sigma_w^4}{N}(\mathbb{E}[\phi'^4] - (\mathbb{E}[\phi'^2])^2)$. The effective finite-width χ_1 is: $\chi_{1,N} = \chi_1 + \frac{\sigma_w^2}{N}(\mathbb{E}[\phi'^4] - (\mathbb{E}[\phi'^2])^2)$. For ReLU: $\mathbb{E}[\phi'^4] = \mathbb{E}[\phi'^2] = 1/2$, so the correction vanishes. □

E Lyapunov Exponent Framework

Definition 14 (Lyapunov Exponent)**.** $\lambda = \ln \chi_1$.

Proposition 15 (Lyapunov Exponent Properties)**.**

1. *$\lambda < 0 \Leftrightarrow$ ordered phase (perturbations decay at rate $e^{\lambda L}$),*
2. *$\lambda = 0 \Leftrightarrow$ edge of chaos,*
3. *$\lambda > 0 \Leftrightarrow$ chaotic phase (perturbations grow at rate $e^{\lambda L}$),*
4. *Depth scale $\xi = 1/|\lambda|$.*

Proof. $\chi_1^L = e^{L \ln \chi_1} = e^{\lambda L}$ and $\xi = 1/|\ln \chi_1| = 1/|\lambda|$. □

F Third-Order Susceptibility χ_3

$$\chi_3 = \sigma_w^2 \int \mathcal{D}z\, [\phi'''(\sqrt{q^*}\, z)]^2\, (q^*)^{3/2} \tag{24}$$

Table 9: Complete bifurcation classification with χ_2 and χ_3.

Activation	σ_w^*	χ_2	χ_3	**Bifurcation**
ReLU	1.4142	0	0	Degenerate
tanh	1.010	0.0382	0.0039	Supercritical
GELU	1.982	2.454	<0.001	Supercritical
SiLU	1.993	0.984	<0.001	Supercritical

For smooth activations, $\chi_3 \ll \chi_2$, confirming supercritical classification by χ_2 alone.

G ResNet and Conv Extensions

G.1 ResNet Mean-Field Extension

Definition 16 (ResNet Variance Propagation)**.** *For a pre-activation residual network with skip scaling α:*

$$q^{(l+1)} = q^{(l)} + 2\alpha\sigma_w^2\, C(q^{(l)}) + \alpha^2(\sigma_w^2\, V(q^{(l)}) + \sigma_b^2) \tag{25}$$

where $C(q) = \mathbb{E}_{z\sim\mathcal{N}(0,q)}[z\,\phi(z)]$ is the cross-covariance.

Proposition 17 (ResNet Susceptibility)**.**

$$\chi_1^{ResNet} = 1 + 2\alpha\sigma_w^2\, C'(q^*) + \alpha^2\sigma_w^2 \int \mathcal{D}z\, [\phi'(\sqrt{q^*}\, z)]^2 \tag{26}$$

Proof. Differentiating (**??**): $dq^{(l+1)}/dq^{(l)} = 1 + 2\alpha\sigma_w^2 C'(q^{(l)}) + \alpha^2\sigma_w^2 \int \mathcal{D}z\, [\phi'(\sqrt{q^*}\, z)]^2$. □

G.2 Conv2d Extension

Proposition 18 (Conv2d Variance Recursion)**.** *For Conv2d with filters $W \sim \mathcal{N}(0, \sigma_w^2/(C_{\text{in}}k^2))$: $q^{(l+1)} = \sigma_w^2 V(q^{(l)}) + \sigma_b^2$, identical to MLPs. Finite-width corrections scale as $O(1/C_{\text{out}})$.*

Proof. The pre-activation $h_c^{(l+1)}(i,j) = \sum_{c'}\sum_{a,b} W_{c,c',a,b}\phi(h_{c'}^{(l)}(i+a, j+b)) + b_c$ is a sum of $C_{\text{in}}k^2$ i.i.d. terms with variance $\sigma_w^2/(C_{\text{in}}k^2) \cdot V(q^{(l)})$, giving total variance $\sigma_w^2 V(q^{(l)}) + \sigma_b^2$. □

H Stochastic Phase Boundary Crossover: Details

Proof of Theorem **??**. At width N, $\hat{\chi}_1 = \sigma_w^2 \frac{1}{N} \sum_{j=1}^{N} [\phi'(h_j)]^2$ is a sample average. By the CLT:

$$\hat{\chi}_1 \sim \mathcal{N}\left(\chi_1,\ \frac{\sigma_w^4}{N}\left(\mathbb{E}[\phi'^4] - (\mathbb{E}[\phi'^2])^2\right)\right) \tag{27}$$

The crossover width in σ_w-space is:

$$\Delta(N) = \frac{\sigma_w^2}{N^{1/2}} \cdot \sqrt{\mathbb{E}[\phi'^4] - (\mathbb{E}[\phi'^2])^2} \cdot \frac{1}{|d\chi_1/d\sigma_w|_{\sigma_w^*}} = \frac{A_\phi}{\sqrt{N}} \tag{28}$$

For ReLU, $\mathbb{E}[\phi'^4] = \mathbb{E}[\phi'^2] = 1/2$, so $\sigma_\chi = 0$—the phase boundary remains sharp at all widths. □

Crossover amplitudes. ReLU: $A_{\text{ReLU}} = 1.4142$. tanh: $A_{\tanh} = 0.0272$. GELU: $A_{\text{GELU}} = 0.6146$. SiLU: $A_{\text{SiLU}} = 0.3936$.

I Extended Activation Support

Table 10: Edge-of-chaos values σ_w^* for all supported activations.

Activation	σ_w^*	**Method**
ReLU	1.4142	Closed-form ($\sqrt{2}$)
LeakyReLU	1.4141	Closed-form ($\sqrt{2/(1+\alpha^2)}$)
tanh	1.0098	Brent + quadrature
GELU	1.9815	Brent + quadrature
SiLU/Swish	1.9926	Brent + quadrature
Mish	1.6610	Brent + quadrature
ELU	1.0067	Brent + quadrature

All edge-of-chaos values validated: $|\chi_1(\sigma_w^*) - 1| < 10^{-6}$.

J Moment-Closure Sensitivity (κ_4): Analytical vs. Empirical

Proposition 19 (Moment-Closure Error Bound)**.** *The analytical κ_4 (computed under Gaussian closure) provides a lower bound on the true κ_4 for all tested activations. The relative error $|\kappa_4^{empirical} - \kappa_4^{analytical}|/\kappa_4^{analytical}$ has median* 18.7% *and mean* 2.81.

The large mean relative error is driven by ReLU, where the analytical $\kappa_4 = 0.5$ (valid for a single Gaussian input layer) underestimates the depth-accumulated value by 10.6×. For smooth activations, the discrepancy is moderate.

Table 11: Moment-closure comparison: analytical vs. empirical κ_4. Measured over 2 widths × 2 σ_w values × 10 layers, 100 MC samples each.

Activation	$\kappa_4^{\text{analytical}}$	$\kappa_4^{\text{empirical}}$	**Ratio**	**Median Rel. Err.**
ReLU	0.500	5.804	11.6×	—
tanh	1.612	1.823	1.13×	13.1%
GELU	2.068	2.443	1.18×	18.1%
SiLU	2.027	2.365	1.17×	16.7%

Practical implications. For smooth activations, the Gaussian closure is adequate for finite-width corrections (errors $< 20\%$). For ReLU, the depth-dependent correction $\kappa_4^{\text{eff}}(l) = \kappa_4 \cdot (1 + 0.09 \ln(l+1))$ partially compensates but does not fully capture the empirical growth. A maximum-entropy moment closure [**?**] could provide better accuracy but at higher computational cost.

K Z3 Verification Details

We encode the mean-field recursion and phase classification logic in Z3's theory of real arithmetic.

Variables. σ_w, σ_b: real-valued hyperparameters; $q_0, q_1, \ldots, q_L$: per-layer variances; χ_1: susceptibility.

Verified properties.

1. **Phase completeness**: $\forall \sigma_w > 0$: exactly one of $\chi_1 < 1$, $\chi_1 = 1$, $\chi_1 > 1$ holds.
2. **Fixed-point existence**: $\sigma_w^2 < 2 \wedge \sigma_b > 0 \Rightarrow \exists q^* > 0$: $q^* = \sigma_w^2 q^*/2 + \sigma_b^2$.
3. **Kaiming–critical equivalence**: $\sigma_b = 0 \Rightarrow (\chi_1 = 1 \Leftrightarrow \sigma_w^2 = 2)$.
4. **Gradient bound monotonicity**: $\chi_1 < 1 \Rightarrow \chi_1^L < \chi_1^{L-1}$ for $L > 1$.
5. **Variance positivity**: $q_0 > 0 \Rightarrow q_l > 0$ for all l.

All properties verified `unsat` (no counterexample) in <1s each.

L Z3 Convergence Verification

We verify that for $\sigma_w^2 < 2$, the ReLU variance recursion converges to q^* (property P8), and that for $\sigma_w^2 > 2$ with $\sigma_b = 0$, iterates diverge (property P9). For $L = 50$ and $\epsilon = 10^{-6}$: convergence verified (`unsat`).

M Z3 Finite-Width Verification

Properties P13–P20 verify that the finite-width correction is well-defined (non-negative for $N > 0$), bounded, and monotonically decreasing in N. For $N \geq 64$, the correction is at most $0.031\sigma_w^4 q^{*2}$, verified for ReLU with $\sigma_w \in [0.1, 10]$.

N Algorithms

Algorithm 2 Variance Map Computation

Require: Activation ϕ, weight variance σ_w^2, bias variance σ_b^2, input variance q_0, depth L
Ensure: Per-layer variances $q_0, q_1, \ldots, q_L$

```
for l = 0 to L − 1 do
    V(q_l) ← ∫ Dz [φ(√q_l z)]^2          ▷ Gauss–Hermite quadrature
    q_{l+1} ← σ_w^2 · V(q_l) + σ_b^2
end for
return (q_0, q_1, ..., q_L)
```

Algorithm 3 Edge-of-Chaos Solver

Require: Activation ϕ, bias variance σ_b^2, tolerance ϵ
Ensure: Critical weight variance σ_w^{*2}

```
Define f(σ_w^2) = σ_w^2 ∫ Dz [φ'(√q*(σ_w^2) z)]^2 − 1
σ_w^{*2} ← Brent(f, [0.01, 100], ε)
return σ_w^{*2}
```

Algorithm 4 Depth-Calibrated Phase Classification

Require: Model parameters $(\sigma_w^2, \sigma_b^2, \phi, L, N)$
Ensure: Phase $\in \{\text{ordered}, \text{critical}, \text{chaotic}\}$

```
q* ← FixedPoint(σ_w^2, σ_b^2, φ)
χ_1 ← σ_w^2 ∫ Dz [φ'(√q* z)]^2
if N < ∞ then
    χ_1 ← χ_1 + (σ_w^2/N)(E[φ'^4] − (E[φ'^2])^2)        ▷ Finite-width correction
end if
χ_min ← 10^(−4/L),   χ_max ← 10^(4/L)                   ▷ Depth-calibrated thresholds
if χ_1 < χ_min then
    return ordered
else if χ_1 > χ_max then
    return chaotic
else
    return critical
end if
```

Algorithm 5 Finite-Width Corrected Variance

Require: Activation ϕ, parameters (σ_w^2, σ_b^2), width N, depth L
Ensure: Corrected per-layer variances $\hat{q}_0, \ldots, \hat{q}_L$

```
Compute mean-field variances q_0, ..., q_L via Algorithm ??
Compute moments μ_4, μ_6 via Gauss–Hermite quadrature
κ_4 ← μ_4/μ_2^2 − 1
for l = 0 to L − 1 do
    q̂_{l+1} ← q_{l+1} + σ_w^4 κ_4 V(q_l)^2/N
end for
return (q̂_0, ..., q̂_L)
```

O Reproducibility

O.1 Computational Environment

All experiments were run on a single machine with:

- CPU: Intel Xeon W-2255 (10 cores, 3.7 GHz)
- GPU: NVIDIA RTX 3090 (24 GB)
- RAM: 128 GB
- OS: Ubuntu 22.04
- Python 3.10, PyTorch 2.0, NumPy 1.24, SciPy 1.10
- Z3 4.12.2, Hypothesis 6.82

O.2 Runtime

O.3 Random Seeds

All stochastic computations use fixed seeds for reproducibility:

- PyTorch: `torch.manual_seed(42)`
- NumPy: `np.random.seed(42)`
- Python: `random.seed(42)`
- CUDA: `torch.cuda.manual_seed_all(42)`

Table 12: Runtime for key computations.

Computation	**Time**	**Hardware**
Edge-of-chaos solver (1 activation)	0.02s	CPU
Phase classification (1 config)	0.05s	CPU
Finite-width correction (1 config)	0.08s	CPU
Training validation (72 configs)	2.1h	GPU
Z3 verification (all properties)	3.1s	CPU
κ_4 Monte Carlo (all configs)	45s	CPU

O.4 Numerical Precision

- Gauss–Hermite quadrature: 100 points (error $< 10^{-12}$ for smooth integrands).
- Brent's method tolerance: 10^{-8}.
- Monte Carlo: 10^4 samples per configuration (standard error < 0.01).
- All floating-point computations in float64.